# Sistema de Felicitaciones de Cumpleaños

Memoria del proyecto BirthdayGreating

Gabriel Alexandru Iacob
2º DAM — IES Francisco de Goya
25 de octubre de 2026

# 1. Objetivos

Se quiere automatizar el envío de felicitaciones de cumpleaños. El programa lee un fichero de texto plano con los datos de los amigos (`last_name, first_name, date_of_birth, email`) y, en la fecha de su cumpleaños, les envía el mensaje:

```
Asunto: ¡Feliz cumpleaños!

¡Feliz cumpleaños, querido/a <first_name>!
```

Objetivos:

- Detectar qué amigos cumplen años en la fecha actual (o en una fecha simulada).
- Enviarles la felicitación con el formato indicado.
- Diseñar el software de forma que se pueda cambiar el **origen de datos** (fichero → SQLite) y el **método de entrega** (email → SMS) sin modificar la lógica principal.
- Incluir un plan de pruebas con tests unitarios, de integración y uso de Mocks.

Mejoras implementadas (las dos propuestas):

- **Años bisiestos:** los nacidos el 29 de febrero reciben la felicitación el 28 de febrero en los años no bisiestos.
- **Recordatorios a terceros:** el resto de contactos recibe un recordatorio del cumpleaños.

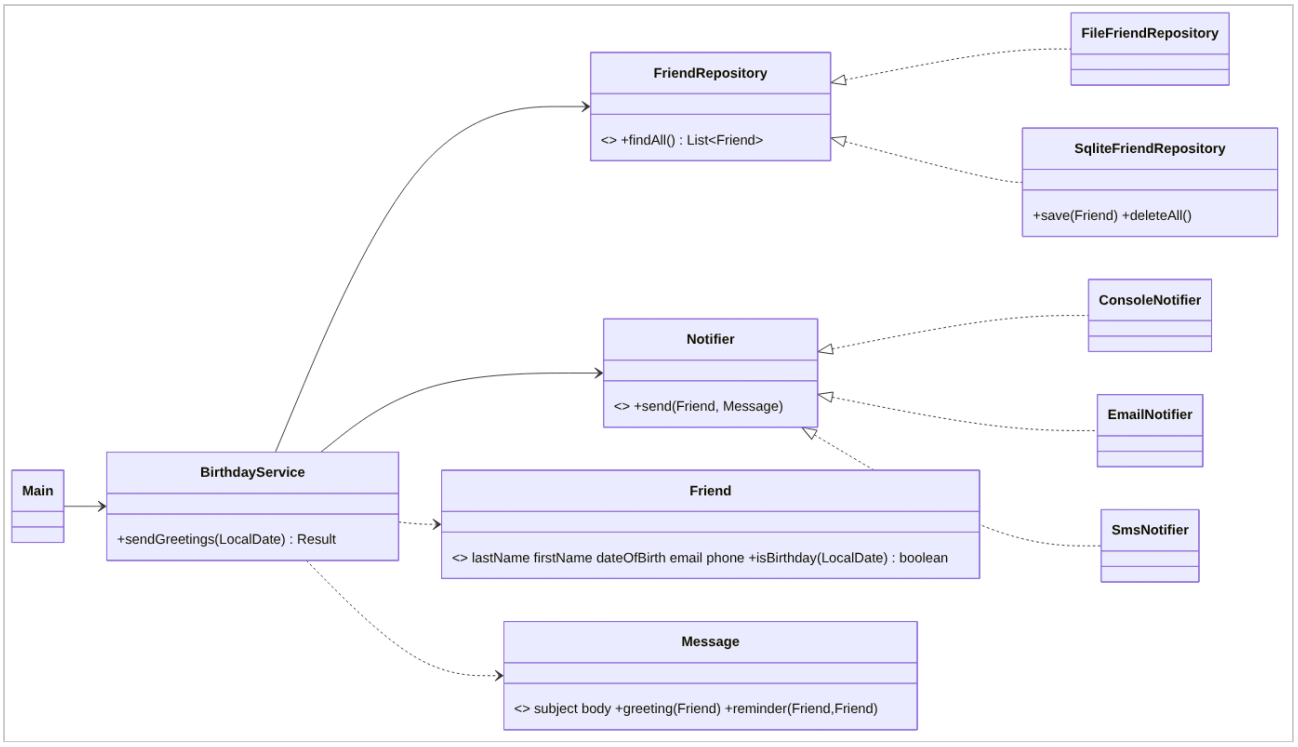
Mejoras adicionales: importación del fichero a una base de datos SQLite, envío por SMS (simulado), envío por consola para pruebas, tolerancia a fallos (si falla un envío continúa con el resto) y resumen final de envíos.

# 2. Herramientas

| **Herramienta** | **Uso** |
|---|---|
| Java 21 | Lenguaje de programación (records, switch expressions, text blocks). |
| Eclipse IDE | Entorno de desarrollo. |
| Maven | Gestión de dependencias y construcción (`pom.xml`). |
| Angus Mail (Jakarta Mail) 2.0.3 | Envío de correos por SMTP (`EmailNotifier`). |
| sqlite-jdbc 3.46.1.3 | Driver JDBC para la base de datos SQLite. |
| JUnit 5 (Jupiter) 5.13.4 | Tests unitarios y de integración. |
| Mockito 5.20.0 | Objetos simulados (Mocks) en los tests. |

# 3. Diseño

## 3.1 Diagrama de clases




La clave del diseño es que `BirthdayService` solo depende de dos interfaces, que recibe por el constructor (inyección de dependencias):

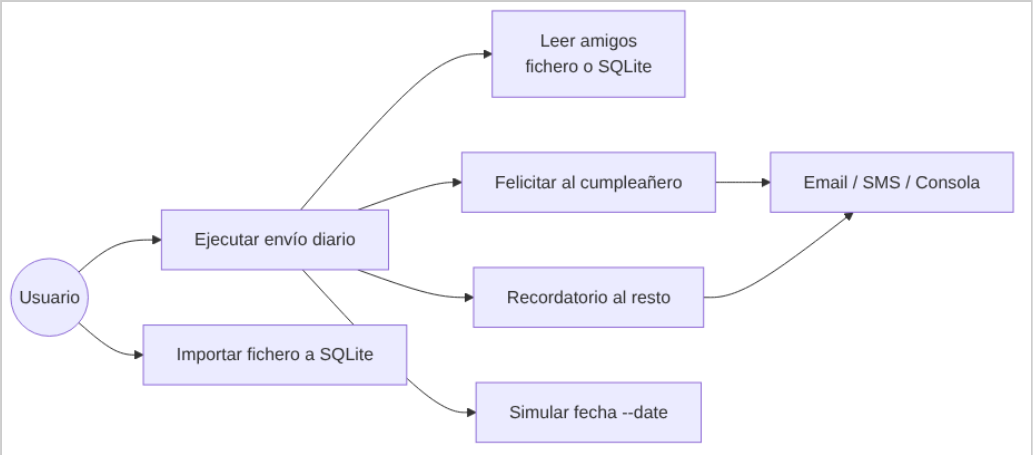
- **FriendRepository** (origen de datos): implementaciones `FileFriendRepository` y `SqliteFriendRepository`.
- **Notifier** (método de entrega): implementaciones `EmailNotifier`, `SmsNotifier` y `ConsoleNotifier`.

Para pasar de fichero a base de datos o de email a SMS basta con crear otra implementación de la interfaz y elegirla en `Main`; el servicio no cambia (principio abierto/cerrado e inversión de dependencias). La fecha se pasa como parámetro a `sendGreetings(LocalDate)`, lo que permite probar cualquier día.

| Paquete | Clases | Responsabilidad |
|---|---|---|
| birthday.model | Friend, Message | Datos del amigo, lógica de cumpleaños (incluido 29/02) y formato de los mensajes. |
| birthday.repository | FriendRepository, FileFriendRepository, SqliteFriendRepository, FriendParser, RepositoryException | Lectura de los amigos desde fichero o SQLite. |
| birthday.notifier | Notifier, EmailNotifier, SmsNotifier, ConsoleNotifier, NotificationException | Envío de los mensajes. |

| | | |
|---|---|---|
| birthday.service | BirthdayService | Lógica principal: felicitaciones y recordatorios. |
| birthday | Main | Lee los argumentos y monta los componentes. |

## 3.2 Casos de uso



# 4. Funcionamiento

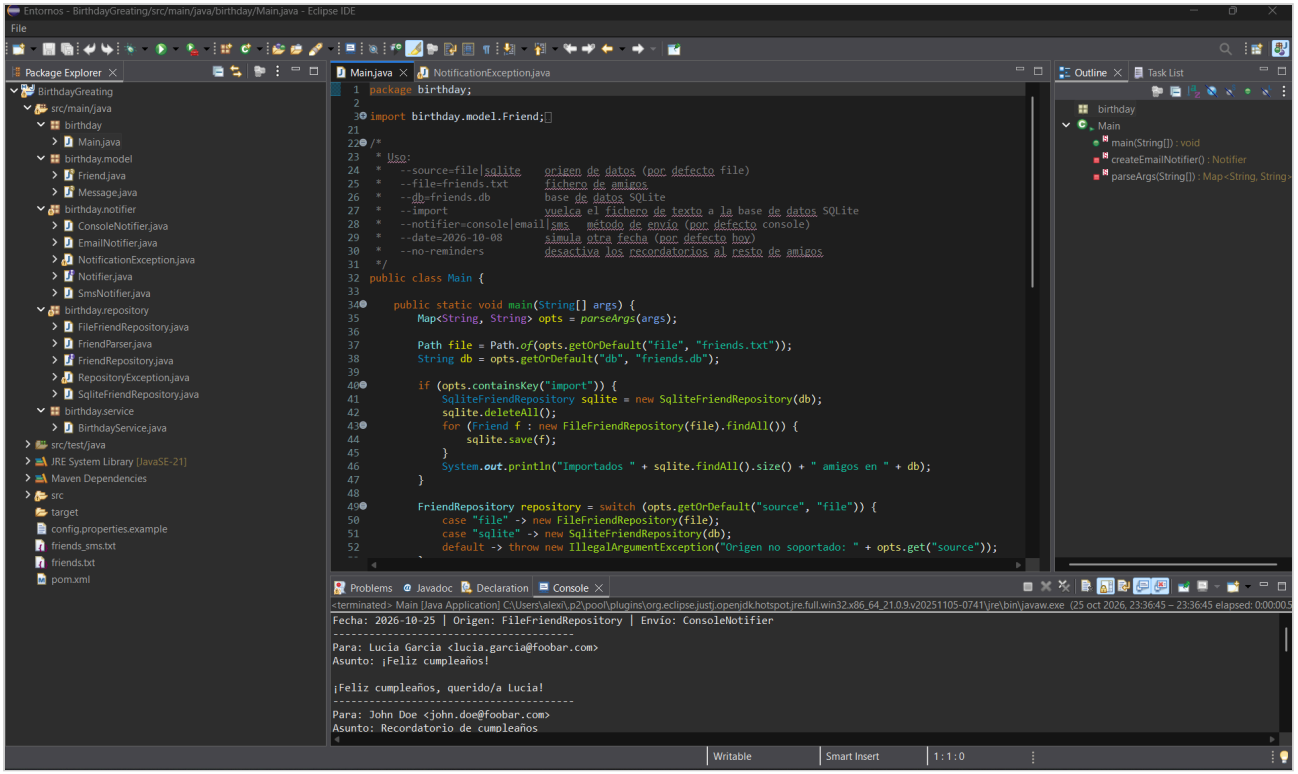
El programa se ejecuta desde `birthday.Main` y admite estos argumentos:

| Argumento | Descripción |
|---|---|
| --source=file\|sqlite | Origen de datos (por defecto file) |
| --file=friends.txt / --db=friends.db | Fichero o base de datos a usar |
| --import | Vuelca el fichero de texto a SQLite |
| --notifier=console\|email\|sms | Método de envío (por defecto console) |
| --date=AAAA-MM-DD | Simula una fecha (por defecto hoy) |
| --no-reminders | Desactiva los recordatorios |

Fichero de entrada `friends.txt`:

```
last_name, first_name, date_of_birth, email
Doe, John, 1982/10/08, john.doe@foobar.com
Ann, Mary, 1975/09/11, mary.ann@foobar.com
Garcia, Lucia, 1999/10/25, lucia.garcia@foobar.com
Leap, Peter, 2000/02/29, peter.leap@foobar.com
```

Ejecución en Eclipse el 25/10/2026 (cumpleaños de Lucia): se envía la felicitación a Lucia y los recordatorios al resto.




### Felicitación con recordatorios (--date=2026-10-08)

```
Fecha: 2026-10-08 | Origen: FileFriendRepository | Envío: ConsoleNotifier
----------------------------------------
Para: John Doe <john.doe@foobar.com>
Asunto: ¡Feliz cumpleaños!

¡Feliz cumpleaños, querido/a John!
----------------------------------------
Para: Mary Ann <mary.ann@foobar.com>
Asunto: Recordatorio de cumpleaños

Querido/a Mary,

Hoy es el cumpleaños de John Doe.
¡No olvides enviarle un mensaje!
----------------------------------------
(... igual para Lucia y Peter ...)
----------------------------------------
Felicitaciones: 1 | Recordatorios: 3 | Errores: 0
```

### Año bisiesto (--date=2027-02-28 --no-reminders)

```
Fecha: 2027-02-28 | Origen: FileFriendRepository | Envío: ConsoleNotifier
----------------------------------------
Para: Peter Leap <peter.leap@foobar.com>
Asunto: ¡Feliz cumpleaños!

¡Feliz cumpleaños, querido/a Peter!
----------------------------------------
Felicitaciones: 1 | Recordatorios: 0 | Errores: 0
```

### Origen SQLite (--import --source=sqlite --date=2026-09-11 --no-reminders)

```
Importados 4 amigos en friends.db
Fecha: 2026-09-11 | Origen: SqliteFriendRepository | Envío: ConsoleNotifier
----------------------------------------
Para: Mary Ann <mary.ann@foobar.com>
Asunto: ¡Feliz cumpleaños!

¡Feliz cumpleaños, querido/a Mary!
----------------------------------------
Felicitaciones: 1 | Recordatorios: 0 | Errores: 0
```

### Envío por SMS (--file=friends_sms.txt --notifier=sms --date=2026-10-25)

```
Fecha: 2026-10-25 | Origen: FileFriendRepository | Envío: SmsNotifier
[SMS -> +34600555666] ¡Feliz cumpleaños, querido/a Lucia!
[SMS -> +34600111222] Querido/a John, Hoy es el cumpleaños de Lucia Garcia. ¡No
olvides enviarle un mensaje!
[SMS -> +34600333444] Querido/a Mary, Hoy es el cumpleaños de Lucia Garcia. ¡No
olvides enviarle un mensaje!
----------------------------------------
Felicitaciones: 1 | Recordatorios: 2 | Errores: 0
```

Para el envío real por correo se copia `config.properties.example` como `config.properties` con los datos SMTP (en Gmail, una contraseña de aplicación) y se ejecuta con `--notifier=email`.

## 5. Estrategia de pruebas

**¿Qué tipo de pruebas escribiría?**

- **Unitarias** de la lógica de fechas (`Friend.isBirthday`), incluido el caso del 29 de febrero, y del formato de los mensajes.
- **Unitarias de los repositorios**: lectura del fichero (cabecera, líneas vacías, líneas mal formadas, fechas incorrectas, fichero inexistente) y guardado/lectura en SQLite usando ficheros temporales (`@TempDir`).
- **Unitarias del servicio** con Mocks: a quién se envía, cuántas veces y con qué contenido.
- **De integración**: fichero real + servicio real, simulando solo el envío.

**¿Usaría Mocks? ¿Dónde?** Sí, con Mockito, en las dependencias externas de `BirthdayService`:

- **Notifier**: no se quiere enviar correos o SMS reales en los tests. El mock permite verificar con `verify` a quién se ha enviado, cuántas veces (`times`, `never`) y capturar el mensaje con `ArgumentCaptor`. También se simula un fallo de envío con `doAnswer`.
- **FriendRepository**: con `when(repository.findAll()).thenReturn(...)` se controla la lista de amigos sin depender de ficheros ni de la base de datos.

```
@Mock FriendRepository repository;
@Mock Notifier notifier;

@Test
void enviaRecordatorioAlRestoDeAmigos() {
    when(repository.findAll()).thenReturn(List.of(john, mary, peter));
    new BirthdayService(repository, notifier, true).sendGreetings(JOHN_BIRTHDAY);
    verify(notifier).send(john, Message.greeting(john));
    verify(notifier).send(mary, Message.reminder(mary, john));
    verify(notifier, times(3)).send(any(), any());
}
```

| Clase de test | Nº | Qué comprueba |
|---|---|---|
| FriendTest | 6 | Cumpleaños el mismo día, otro día, 29/02 en año bisiesto y no bisiesto, 28/02, formato de mensajes |
| FileFriendRepositoryTest | 6 | Lectura correcta, teléfono opcional, fichero vacío, línea mal formada, fecha incorrecta, fichero inexistente |
| SqliteFriendRepositoryTest | 3 | Guardar y recuperar, base de datos vacía, borrado |
| BirthdayServiceTest (Mockito) | 6 | Solo se felicita al cumpleañero, contenido del mensaje, nadie cumple, recordatorios, 29/02, fallo de envío |
| BirthdayIntegrationTest | 1 | Fichero real + servicio real con notificador simulado |
| NotifierTest | 3 | Salida por consola, SMS al teléfono, SMS sin teléfono |
| **Total** | **25** | |

## 6. Conclusiones

Se han cumplido los requisitos mínimos: el programa lee el fichero, detecta los cumpleaños del día y envía la felicitación con el formato pedido. Gracias a las interfaces `FriendRepository` y `Notifier`, el origen de datos y el método de envío se cambian sin tocar la lógica, y esto se ha demostrado con SQLite y con SMS. Se han implementado las dos mejoras (años bisiestos y recordatorios) y un plan de pruebas con 25 tests usando JUnit 5 y Mockito.

Posibles mejoras:

- Programar la ejecución diaria automática (cron / Programador de tareas de Windows).
- Integrar un proveedor real de SMS (Twilio, Vonage).
- Evitar envíos duplicados si se ejecuta dos veces el mismo día (registro de envíos).
- Agrupar varios cumpleaños del mismo día en un solo recordatorio.
- Mensajes en HTML y plantillas personalizables.